I'm optimizing this specifically for **your goal**:

> **Learn enough ML to understand how ML works + build ML projects confidently → then move to Deep Learning/AI → GenAI → RAG → Vector DB → Agents.**

So I'm **not** optimizing for becoming a data-science/ML research specialist.

🔴 Videos I would SKIP

These are the ones where I think the time spent is not worth it for your current goal.

Fundamentals / setup

- **11 — What are Tensors** → 🔴 Skip for now
  You'll learn tensors properly when you start Deep Learning/PyTorch.
- **12 — Installing Anaconda / Jupyter / Google Colab** → 🔴 Skip if you already know your environment.
- **22 — Pandas Profiling** → 🔴 Skip
  Nice tool, but not important ML knowledge.

Feature engineering

- **32 — Binning and Binarization** → 🔴 Skip
- **38 — Missing Indicator / Random Sample Imputation** → 🔴 Skip
- **39 — KNN Imputer** → 🔴 Skip
- **40 — MICE / Iterative Imputer** → 🔴 Skip
- **44 — Percentile Method / Winsorization** → 🔴 Skip

You should know **what missing values and outliers are** and the common ways of handling them. You don't need every possible technique.

PCA mathematics

- **48 — PCA Part 2: Problem Formulation & Step-by-Step Mathematical Solution** → 🔴 Skip

Do **47 and 49** instead:

> 47 = understand the intuition
> 49 = see how to implement it

That's enough for your goal.

Regression mathematics

- **51 — Simple Linear Regression Mathematical Formulation / Coding from Scratch** → 🔴 Skip

- **54 — Multiple Linear Regression Mathematical Formulation** → 🔴 Skip
- **55 — Multiple Linear Regression Code From Scratch** → 🔴 Skip
- **56 — Linear Regression Assumptions** → 🟡 I'd skim rather than completely ignore

You should understand regression, but you don't need to implement the algorithm from scratch mathematically.

Gradient Descent

- **57 — Gradient Descent From Scratch / Animation** → 🟡 Don't completely skip; watch conceptually
- **58 — Batch Gradient Descent Code** → 🔴 Skip
- **59 — Stochastic Gradient Descent** → 🔴 Skip
- **60 — Mini-Batch Gradient Descent** → 🔴 Skip

Important: Learn what gradient descent *does*. You don't need three separate implementation videos.

Regularization mathematics

- **64 — Ridge Part 2: Mathematical Formulation** → 🔴 Skip
- **65 — Ridge Part 3: Gradient Descent** → 🔴 Skip
- **68 — Why Lasso Creates Sparsity?** → 🔴 Skip

But **63 Ridge intuition**, **66 Ridge key points**, **67 Lasso**, and **69 ElasticNet** are useful enough to understand quickly.

Logistic Regression mathematics

- **73 — Logistic Regression Part 4: Loss Function / Maximum Likelihood / Binary Cross Entropy** → 🟡 **Skim**, don't deeply study
- **74 — Derivative of Sigmoid Function** → 🔴 Skip
- **75 — Logistic Regression From Scratch / Gradient Descent** → 🔴 Skip

You should know:

Sigmoid → probability → threshold → class

That's enough at this stage.

Naive Bayes — biggest time-saving opportunity

There are **9 videos** here, and you don't need all of them.

🔴 Skip:

- **82 — Conditional Probability**
- **83 — Independent Events**

- **84 — Mutually Exclusive Events**
- **85 — Bayes Theorem**
- **86 — Bayes Theorem Problem**
- **88 — Mathematics Behind Naive Bayes**

That's a lot of probability theory for relatively little benefit to your intended path.

🟢 Watch:

- **87 — Naive Bayes Intuition**
- **89 — Simple Example Code**
- **90 — Handling Numerical Data**

So:

> **87 → 89 → 90**

is enough.

SVM

You definitely don't need the mathematical deep dive.

🔴 Skip:

- **93 — Mathematics of SVM: Hard Margin**
- **94 — Mathematics of SVM: Soft Margin**

🟡 Watch:

- **92 — SVM Geometric Intuition**
- **95 — Kernel Trick Geometric Intuition**
- **96 — Kernel Trick Code**

You should simply understand:

> SVM finds a good separating boundary → margin → kernels allow nonlinear separation.

Done.

Decision Trees

Don't skip the fundamentals.

🟢 Do:

- **97 — Decision Tree Geometric Intuition**
- **98 — Decision Tree Hyperparameters**
- **99 — Regression Trees**

🔴 Skip:

- **100 — dtreeviz visualization**

It's a useful visualization tool, but not important ML knowledge.

Ensemble Learning

Here's where you can save **a lot of time**.

🟢 Do:

- **101 — Introduction to Ensemble Learning**
- **105 — Bagging Introduction**
- **108 — Random Forest**
- **109 — Why Random Forest works**
- **110 — Bagging vs Random Forest**
- **111 — Random Forest Hyperparameters**
- **112 — Random Forest Hyperparameter Tuning**
- **114 — Feature Importance**
- **119 — Bagging vs Boosting**
- **120 — Gradient Boosting**

🔴 Skip:

- **104 — Voting Regression**
- **107 — Bagging Regressor**
- **117 — AdaBoost Code From Scratch**
- **118 — AdaBoost Hyperparameters**

🟡 Skim:

- **102–103 — Voting**
- **106 — Bagging Classifier**
- **113 — OOB Score**

- **115–116 — AdaBoost**

You don't need to master every ensemble technique.

The later part

For the remaining videos, I would **not aggressively skip** XGBoost/K-Means/DBSCAN.

In particular:

🟢 Don't skip:

121 — Gradient Boosting Mathematics
Actually, for you I'd change this to 🟡 **skim**, because it's mathematics-heavy.

122 — Gradient Boosting Classification → 🟢 Do

123 — XGBoost Introduction → 🟢 **MUST DO**

124 — XGBoost Regression → 🟢 Do

125 — XGBoost Classification → 🟢 Do

126 — Maths Behind XGBoost → 🔴 **SKIP**

127 — Stacking and Blending → 🔴 Skip

128 — K-Means Geometric Intuition → 🟢 Do

129 — K-Means Python → 🟢 Do

130 — K-Means From Scratch → 🔴 Skip

131 — Hierarchical Clustering → 🟡 Skim

132 — DBSCAN → 🟢 Do

133 — Imbalanced Data / SMOTE → 🟢 Do

134 — Optuna / Bayesian Optimization → 🟡 Skim

🔥 Your final SKIP list

If you want a simple list to bookmark:

11, 12, 22, 32, 38, 39, 40, 44, 48, 51, 54, 55, 58, 59, 60, 64, 65, 68, 74, 75, 82, 83, 84, 85, 86, 88, 93, 94, 100, 104, 107, 117, 118, 126, 127, 130

That's **37 videos skipped out of 134**.

But there's an even more important distinction:

🟢 Your "core ML" path

1–10 → 13–21 → 23–31 → 33–37 → 41–43 → 45–47 → 49–50 → 52–53 → 56–57 → 61–63 → 66–67 → 69–73 → 76–81 → 87 → 89–92 → 95–99 → 101 → 105–106 → 108–116 → 119–125 → 128–129 → 131–134

You **don't have to memorize everything** in those videos.

The goal is to reach the point where you can comfortably look at a project and understand:

Data → preprocessing → features → train/test → model → training → evaluation → tuning → prediction

Then you're ready to move forward.

And honestly, **once you finish this curated path, I would stop doing classical ML and move toward Deep Learning/Neural Networks rather than trying to squeeze every last bit out of the 100 Days playlist.** That's much more aligned with your eventual **GenAI + RAG + Agents** direction.

You said:

What does skim means

ChatGPT said:

Skim means **watch it quickly just to understand the main idea**, without trying to master every detail.

For your ML learning, if I say 🟡 **SKIM**, do this:

- ▶️ Watch the video at **1.5×–2×**
- 🧠 Understand **what the concept is**
- 🎯 Understand **why/when it is used**
- 💻 Look at the basic code/implementation
- ❌ Don't memorize formulas
- ❌ Don't pause repeatedly for difficult mathematics
- ❌ Don't practice the mathematical derivation

Example: SVM Mathematics

If a video explains complicated SVM equations:

Skim:

> "Okay, SVM finds a separating boundary with maximum margin. Kernel trick helps with nonlinear data."

Then move on.

You don't need to spend **2 hours understanding the mathematical derivation**.

Simple rule for you

🟢 **DO** = Learn properly + understand + practice
🟡 **SKIM** = Understand the idea + basic implementation → move on
🔴 **SKIP** = Don't watch; save the time for more important topics

For **your goal**, this approach will let you cover the important ML concepts much faster.

You said:

Will this make me work on projects of ml

ChatGPT said:

Yes — **but with one important point:** the curated playlist will give you the **ML knowledge needed to build projects**, but watching videos alone won't make you good at projects.

For your goal, I would do:

CampusX curated videos → small ML projects → Deep Learning → GenAI/RAG

After the curated ML portion, you should be able to build projects like:

- 🏠 **House Price Prediction** — regression
- 💳 **Credit Risk / Loan Approval** — classification
- 📧 **Spam Detection** — Naive Bayes / Logistic Regression
- 👥 **Customer Segmentation** — K-Means
- 🚗 **Used Car Price Prediction** — regression
- 🛒 **Customer Churn Prediction** — classification
- 📊 **Employee Attrition Prediction** — classification
- 🌆 **City Noise/Sound Classification** — your **EchoMap** idea
- 📰 **News/Article Classification** — classical ML + NLP
- 🎯 **Recommendation/Clustering projects**

And importantly, you **don't need to know every algorithm** to build these.

For example, for a real project:

```
Dataset
   ↓
EDA
   ↓
Clean missing values
   ↓
Feature Engineering
   ↓
Train/Test Split
```

```
   ↓
Preprocessing Pipeline
   ↓
Try 2–4 Models
   ↓
Compare Metrics
   ↓
Tune Best Model
   ↓
Save Model
   ↓
Make Predictions
```

That's the **actual ML workflow** you want to become comfortable with.

One thing I'd change for you

Since you specifically want to **work on ML projects**, don't wait until Video 134 to start.

After you finish roughly the **regression + classification fundamentals**, start making projects **alongside the playlist**.

For example:

Regression → 1 regression project
Classification → 1 classification project
Clustering → 1 clustering project
Ensemble/XGBoost → 1 stronger project

Then you'll learn much faster because you'll actually use the concepts.

And when you later move into **RAG, vector databases, embeddings, agents, etc.**, you won't feel like ML is a completely foreign subject—you'll understand the underlying ML workflow.

So yes: **this curated path + 3–5 properly built ML projects is enough for the level of classical ML you currently need.**

You said:

For ai (gen ai rag etc)
Do i need to learn dl?

ChatGPT said:

Yes — **but not deeply**.

For your goal of **AI → GenAI → RAG → Vector DBs → Agents**, I would recommend learning **DL fundamentals**, but **not becoming a deep-learning specialist**.

Your ideal path

```
Classical ML
   ↓
DL fundamentals  ← YOU NEED THIS
   ↓
Neural Networks
   ↓
Embeddings
   ↓
Transformers
   ↓
LLMs
   ↓
RAG
   ↓
Vector Databases
   ↓
AI Agents
```

What you should learn from DL

🟢 Definitely learn

- What neural networks are
- Perceptron
- Layers, neurons, weights, biases
- Forward propagation
- Activation functions
- Loss functions
- Backpropagation — **intuition**
- Gradient descent
- Training vs validation vs test
- Overfitting / regularization
- Batch, epochs, learning rate
- CNN basics — understand what they do
- RNN/LSTM basics — understand what they were used for
- **Embeddings** ⭐
- **Attention mechanism** ⭐⭐⭐
- **Transformers** ⭐⭐⭐
- Encoder vs decoder
- Self-attention
- Positional encoding
- Why Transformers enabled modern LLMs

🔴 You don't need to go very deep

For your intended path, don't spend months on:

- Deriving backpropagation mathematically
- Manually calculating gradients
- Detailed CNN mathematics
- Detailed RNN mathematics
- Implementing every neural network from scratch
- Advanced optimization mathematics
- Research-level architectures
- Proving neural-network equations

You should understand **what is happening and why**, rather than becoming someone who can derive everything mathematically.

The most important transition

The part I'd take **very seriously** is:

Neural Networks → Embeddings → Attention → Transformers → LLMs

Because this connects classical ML/DL knowledge directly to what you actually want to build.

For example, when you eventually use a vector database in a RAG system:

```
Document
   ↓
Embedding Model
   ↓
Vector
   ↓
Vector Database
   ↓
Similarity Search
   ↓
Relevant chunks
   ↓
LLM
   ↓
Answer
```

Understanding **embeddings** and the basic idea of how neural networks learn representations makes this much easier to understand instead of just blindly using an API.

For you specifically

I would **not** do:

> 100 Days ML → 6 months of mathematical DL → finally GenAI

I'd do:

> **Curated ML → ML projects → DL fundamentals → Transformers/LLMs → RAG → Vector DB → Agents**

That keeps you aligned with your actual goal of becoming a **practical AI/GenAI engineer**, rather than spending a huge amount of time becoming an ML/DL researcher.